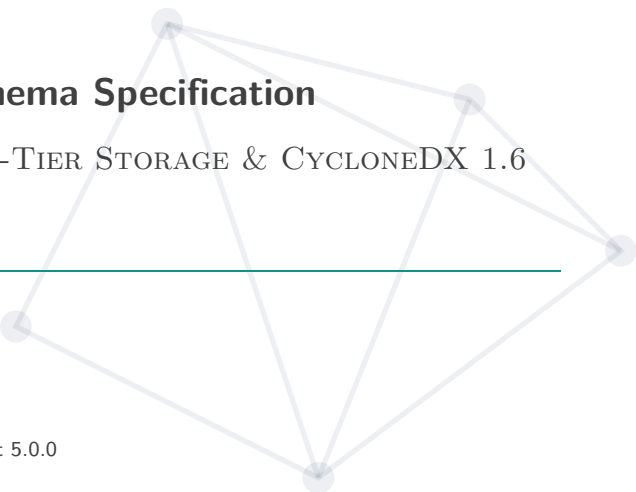


AI-BOM Architecture & Schema Specification



EVIDENCE-BASED IDENTITY, TWO-TIER STORAGE & CYCLONEDX 1.6
ML-BOM

AI-BOM - SCA

Ngày 2 tháng 9 năm 2026

AI Software Bill of Materials Engine 0.1.0 · Product 5.0.0

Tổng Quan Vấn Đề & Nguyên Tắc Thiết Kế

Crawl Dữ Liệu Hugging Face & Cache SQLite

Quy Tắc Đánh Giá Độ Tin Cậy & Xử Lý Xung Đột

12 Tình Huống Thực Tế Khi Quét Model & JSON Mẫu

Cấu Trúc JSON Đầu Ra: Chuẩn Quốc Tế & Nội Bộ

Đọc Header Binary, Chống Mã Độc & Kết Quả Test

Tổng Quan Vấn Đề & Nguyên Tắc Thiết Kế

Tóm Tắt Giải Pháp (Executive Summary)

! Thực Trạng

- 0 có registry chuẩn (như npm/Cargo) → model upload tự do.
- File weights bị re-upload khắp nơi → metadata lệch pha.
- Model Card tác giả tự gõ tay → 0 ai kiểm chứng.

🛡 Đề Xuất

- **4 mức tin cậy:** Đo trực tiếp từ file binary > Lời khai trên web.
- **SQLite 2 tầng:** Tải data 1 lần, scan 100% offline (0 cần mạng).
- **CycloneDX 1.6:** Chuẩn hóa quốc tế, minh bạch evidence.

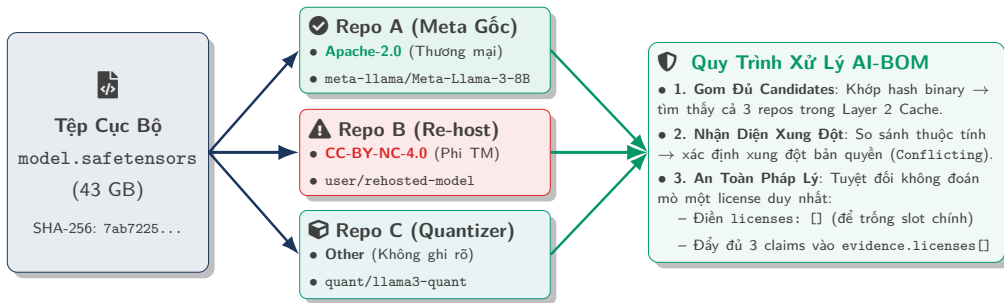
✔ Dữ Liệu Thực Nghiệm

Đã kiểm thử trên **49,992 models** (798,682 file hashes, DB nén 912 MB).

So Sánh: SBOM Phần Mềm Truyền Thống vs AI-BOM

Tiêu chí	Software SBOM (npm/Cargo)	AI-BOM (Hugging Face/GGUF)
1. Phân phối	Registry tập trung, version bất biến	Hub mở, ai cũng fork/re-upload được
2. Bản chất File	Source code / text đọc được	Matrix weights nhị phân lớn (GB–TB)
3. Quan hệ Hash	1 Version ↔ 1 SHA-256 duy nhất	1 Hash → 1,037 Repos (Trùng lặp lớn)
4. Metadata	Khai báo chuẩn qua lockfile	YAML Model Card tự gõ tay
5. License	SPDX rõ ràng, chuẩn 1–1	Tranh chấp license giữa repo gốc và fork

Sơ Đồ Trực Quan: Cơ Chế Nhận Diện & Xử Lý Xung Đột Đa Repo (1-N)



💡 Nguyên Tắc An Toàn Pháp Lý

Khi một file nhị phân trả về nhiều repos có bản quyền mâu thuẫn, scanner giữ sự trung thực tuyệt đối của bằng chứng: để trống kết luận phán đoán và cung cấp đủ dữ liệu đối soát để bảo vệ khách hàng.

5 Nguyên Tắc Thiết Kế Cốt Lõi



1. Định Danh Theo File Thực Tế

Tính danh tính từ hash binary thật, không tin vào đường dẫn file.



2. Rõ Ràng Nguồn Gốc

Mọi thông tin đều ghi rõ lấy từ đâu (local_binary, hf_card).



3. Không Chắc Thì Từ Chối - Tuyệt Đối Không Đoán Mò

Thiếu bằng chứng hoặc mâu thuẫn → để trống hoặc ghi withheld, không tự bịa dữ liệu.



4. Tách Bằng Chứng & Kết Luận

Lưu cả dữ liệu thô (claims[]) và kết luận (concludedValue).



5. Kết Quả Tất Định

Cùng input → output JSON trùng 100% (RFC 9562 UUIDv8).

6 Nguyên Tắc Bất Biến Trong Thiết Kế Hệ Thống

#	Nguyên Tắc Bất Biến	Mục Đích & Hành Vi Kỹ Thuật
1	Quét Offline 100%	Rebuild toàn bộ Cache L2 từ Raw L1 với 0 request mạng .
2	Minh Bạch Bằng Chứng	Output JSON luôn chứa cả <code>claims[]</code> thô và kết luận chốt.
3	Không Đoán Mò	Dữ liệu mâu thuẫn → giữ nguyên tranh chấp, 0 tự ý chọn 1 bên.
4	Core Không Dính Mạng	Crate <code>aibom-core</code> hoàn toàn 0 chứa code gọi mạng.
5	Loại Bỏ File Rỗng	Bỏ qua hash file 0-byte khi tính danh tính weight set của model.
6	Đối Chiếu An Toàn	Chỉ nạp config ngoài khi thư mục chỉ chứa đúng 1 model.

Crawl Dữ Liệu Hugging Face & Cache SQLite

Số Liệu Thực Tế Khi Crawl 50.000 Models (data/poc-50k)

Chỉ số đo đạc	Dataset 20k	Dataset 50k (Hiện tại)	 Nhận Xét Thực Tế
Tổng số Models	19,998	49,992 / 50,000	▪ 5.11% hash xuất hiện ở > 1 repo → hiện tượng trùng file là bản chất của AI Hub. ▪ 65.1% repo chứa > 1 nhóm weights → bắt buộc phải tách nhỏ theo từng nhóm.
Tổng số File Hashes	268,052	798,682 hashes	
File Weights	267,979	705,158 (93.85%)	
Distinct Weight Sets	19,235	47,854 sets	
Hash trùng đa Repo	6.10%	5.11% (max 1,037)	
Repo đa nhóm Weight	57.4%	65.1% repos	
Dung lượng DB L2	360 MB	912 MB (18 KB/model)	

Những "Bẫy Dữ Liệu" Trên Hugging Face Hub

1. README Lộn Xộn

Tác giả dán cả bài báo vào YAML → file phình to hàng chục MB dữ liệu rác.

2. File Dùng Chung Trùng Hàng Nghìn Repo

File `tokenizer.json` xuất hiện ở **1,037 repos** với 12 license khác nhau.

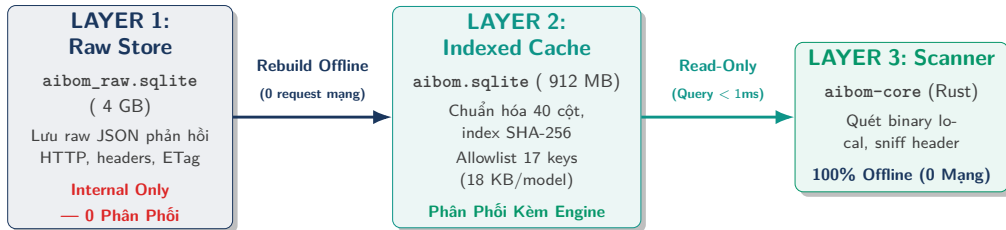
3. Mâu Thuẫn Trong Cùng 1 Repo

Cùng 1 model: Tags ngoài ghi Apache-2.0 nhưng Card trong ghi MIT.

4. 1 Repo Chứa Nhiều Model Con

1 repo chứa 7 file GGUF lượng tử hóa khác nhau (Q2, Q4, FP16).

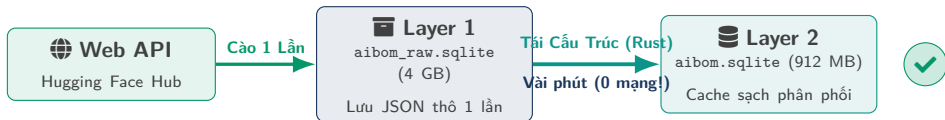
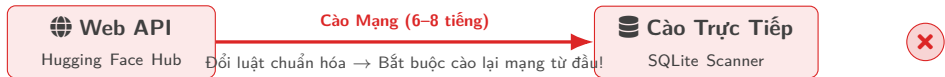
Kiến Trúc SQLite 2 Tầng: Tách Biệt Raw Data & Cache



💡 Lợi Ích Của Thiết Kế 2 Tầng

- **Gọn nhẹ:** DB phân phối kèm scanner chỉ nặng 912 MB (18 KB/model).
- **Rebuild siêu nhanh hoàn toàn offline:** Khi đổi rule normalize, rebuild L2 từ L1 trong vài phút, 0 cần crawl lại mạng.

Sơ Đồ Trực Quan: Chu Trình "Cào Dữ Liệu Một Lần, Tái Cấu Trúc Mỗi Lần"



Ý Nghĩa Kiến Trúc

Tách biệt hoàn toàn tầng lưu trữ nguyên thủy (Raw Ingestion) và tầng lập chỉ mục (Clean Projection). Giúp kỹ sư thử nghiệm hàng chục bộ quy tắc chuẩn hóa mà không gây nghẽn mạng hay bị khóa API.

Bộ Lọc 17 Trường Quan Trọng Từ Model Card (Allowlist)

☰ 17 Trường Cần Thiết Được Giữ Lại

Chỉ giữ đúng 17 trường có giá trị kiểm toán:

- | | |
|-----------------|----------------------|
| 1. license | 10. model_name |
| 2. library_name | 11. license_name |
| 3. pipeline_tag | 12. license_link |
| 4. tags | 13. widget |
| 5. datasets | 14. co2_eq_emissions |
| 6. metrics | 15. fine_tuned_from |
| 7. base_model | 16. thumbnail |
| 8. language | 17. model_type |
| 9. inference | |

⚡ Hiệu Quả Nén Dữ Liệu

- Loại bỏ văn bản bài báo rác → dung lượng còn **18.0 KB / model**.
- Toàn bộ 50.000 models lưu gọn trong **912 MB SQLite**.

Bảng Model & Xử Lý Model Khai Báo License "Other"

```
CREATE TABLE model (  
  repo_id TEXT PRIMARY KEY, author TEXT, model_name TEXT, sha TEXT,  
  last_modified TEXT, private INTEGER, gated INTEGER, disabled INTEGER,  
  pipeline_tag TEXT, library_name TEXT, model_type TEXT, model_architecture TEXT,  
  license_spdx TEXT,  
  license_name TEXT, -- Lưu tên license gốc khi khai báo "other"  
  license_link TEXT, -- Lưu link điều khoản license gốc  
  downloads INTEGER, likes INTEGER, created_at TEXT,  
  config_json TEXT, card_data_json TEXT, card_text_sha256 TEXT,  
  weight_set_count INTEGER, weight_file_count INTEGER, total_weight_bytes INTEGER  
  -- Tổng cộng 40 cột chuẩn hóa  
);
```

✓ Chuẩn Hóa 70.0% Model Khai Báo License "Other"

Tách riêng 2 cột license_name & license_link → chuẩn hóa 3,466 / 4,950 models khai other (đạt 70.0%).

Bảng File, Weight Set & Ảnh Xạ FileRole

```
CREATE TABLE model_file (  
    repo_id TEXT, path TEXT, size_bytes INTEGER, sha256 TEXT, lfs_sha256 TEXT,  
    role TEXT, -- weight / config / tokenizer / license / doc / other (DDL)  
    PRIMARY KEY (repo_id, path)  
);  
  
CREATE TABLE weight_set (  
    weight_set_sha256 TEXT PRIMARY KEY, -- Hash tổng hợp của weight group  
    repo_id TEXT, member_count INTEGER, total_bytes INTEGER  
);
```

Ảnh Xạ 6 Role DDL Thô → 5 Cấp FileRole Trong Scanner

- DB DDL (6 role thô): weight, config, tokenizer, license, doc, other.
- Scanner Engine (5 FileRole): Weight (measured), Auxiliary (khóa trần heuristic cho config/tokenizer/license/doc), Media, Archive, Other.

Bảng Ghi Nhận Thiếu Sót (Gaps) & Tối Ưu DB

```
CREATE TABLE harvest_gap (  
    repo_id TEXT NOT NULL,  
    field TEXT NOT NULL,  
    provider TEXT NOT NULL,  
    pointer TEXT NOT NULL DEFAULT '', -- Tránh lỗi NULL của SQLite  
    PRIMARY KEY (repo_id, field, provider, pointer)  
);
```



Khóa Chính 4 Cột Tránh Lọc Bản Ghi Trùng

SQLite coi NULL != NULL → dùng pointer NOT NULL DEFAULT '' để bảo đảm lọc trùng chính xác 100%.

* Đóng Băng File SQLite Trước Khi Ship

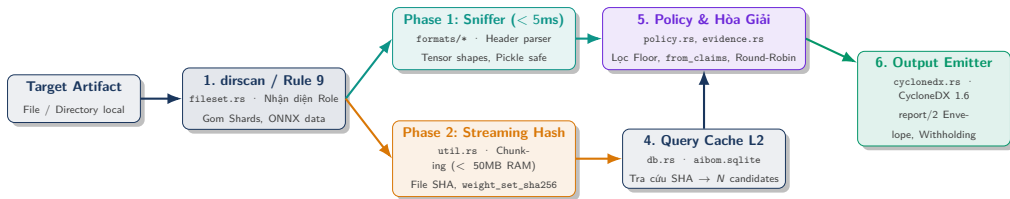
Lệnh tối ưu trước khi release:

```
PRAGMA journal_mode = DELETE;  
PRAGMA wal_autocheckpoint = 0;  
VACUUM;
```

- 1 file duy nhất, read-only, không sinh file phụ -wal.

Quy Tắc Đánh Giá Độ Tin Cậy & Xử Lý Xung Đột

Sơ Đồ Luồng Thực Thi Runtime Của aibom-core



⚙️ Quy Trình Thực Thi Khép Kín 100% Offline

Dữ liệu di chuyển tuần tự qua 7 module Rust độc lập, không chia sẻ trạng thái toàn cục, bảo đảm tính tất định và an toàn bộ nhớ.

4 Mức Độ Tin Cậy Của Dữ Liệu (Evidence Levels)

heuristic (0) < declared (1) < verified (2) < measured (3)

Cấp	Tên Cấp	Định Nghĩa	Cách Thu Thập Trong Engine
0	Heuristic	Phỏng đoán ngoại quan / regex tên file	Đuôi .safetensors → đoán framework
1	Declared	Tác giả tự khai trên web (chưa verify)	Model Card ghi license: mit
2	Verified	2 nguồn độc lập xác thực lẫn nhau	Header file khớp với Model Card
3	Measured	Đo trực tiếp từ file binary	Đọc tensor header ra đúng số params

Tại Sao Dùng Thang Thứ Tự (Ordinal Scale)?

Thang thứ tự (Ord trong Rust) giúp so sánh bằng chứng hoàn toàn bất định, 0 dùng điểm số xác suất mơ hồ.

Ví Dụ Thực Tế: 4 Cấp Độ Khi Tìm Tham Số Model Meta-Llama-3-8B

🔍 Cấp 0: Heuristic & Cấp 1: Declared

- **Cấp 0 (Heuristic):** Nhìn tên file llama-3-8b.gguf → Phỏng đoán sơ bộ là model khoảng 8 tỷ tham số (8B).
- **Cấp 1 (Declared):** Đọc Model Card trên HF Web → Tác giả tự gõ tay vào README: param_count: 8B.

✅ Cấp 2: Verified & Cấp 3: Measured

- **Cấp 2 (Verified):** Web ghi 8B, file đo ra 8.03B (khớp sai số cho phép) → Đạt cấp **Verified** (Đã kiểm chứng).
- **Cấp 3 (Measured):** Parse tensor header → Đếm chính xác **8,030,261,248 params** (Đo lường vật lý tuyệt đối).

Ảnh Dụ Trực Quan: Kiểm Tra Cân Nặng Hành Lý Sân Bay

1. Tự Khai (Declared)

Hành động sân bay:

Khách dán mác giấy lên vali ghi: "*Vali nặng 7 kg*".

Tương đương AI-BOM:

Tác giả tự gõ vào Model Card: param_count: 7B.

Đặc tính: Lời tự khai một phía, có thể gõ nhầm (70B) hoặc làm tròn thương mại.

2. Xác Thực (Verified)

Hành động sân bay:

Nhân viên nhìn cân (6.74 kg) $\approx$ mác (7 kg) $\rightarrow$ Đóng dấu **ĐÃ XÁC NHẬN**.

Tương đương AI-BOM:

Khai báo và đo đạc khớp nhau trong dung sai $\rightarrow$ Verified.

Đặc tính: Hai nguồn độc lập xác nhận lẫn nhau.

3. Đo Đặc Vật Lý

Hành động sân bay:

Đặt vali lên cân điện tử, đồng hồ hiện: **6.74 kg**.

Tương đương AI-BOM (Measured):

Engine mở file nhị phân, đếm từng tensor: **6.74B params**.

Đặc tính: Sự thật vật lý tất định 100%, đo ở máy nào cũng ra đúng con số này.

Khi Có Mâu Thuẫn: Bàn Cân Điện Tử Luôn Là Chân Lý!

Nếu mác dán ghi **70 kg** nhưng cân điện tử chỉ hiện **6.74 kg** $\rightarrow$ Bỏ lời khai mác giấy, chốt theo số cân thực tế (**Measured thắng tuyệt đối**), đồng thời lập biên bản ghi nhận mâu thuẫn để đối soát.

Khi Dữ Liệu Khai Báo Mâu Thuẫn Với Đo Đạc Thực Tế

Cấp Bằng Chứng	Cách Thu Thập	Ví Dụ Trong Thực Tế
1. Declared	Tác giả tự khai trong Model Card	Card ghi: <code>param_count: 7B</code>
2. Verified	2 nguồn độc lập khớp nhau trong sai số	Khớp dung sai ($6.74B \approx 7B$) → Verified
3. Measured	Đo cấu trúc tensor từ file binary	Parse header: 6,738,415,616 params
Mâu thuẫn	Card khai 70B nhưng file chỉ có 6.74B	Measured thắng tuyệt đối , lưu log audit

⚠ 1. Sai Lệch Khai Báo (Model Card)

- Repo fork đổi tên model thành `super-llama-70b`.
- Model Card khai báo: `parameters: 70B`.
- File nhị phân thực tế chỉ có 6.74B params.

🛡 2. Phán Quyết Của Scanner

- Parse header tensor → xác định **6.74B params**.
- **Measured > Declared** → kết luận 6.74B vào BOM.
- Ghi log: `rejected_claim: 70B (hf_card)`.

3 Trục Đánh Giá Dữ Liệu Cốt Lõi

1. EvidenceClass

Mức độ tin cậy (4 cấp):

- heuristic (0)
- declared (1)
- verified (2)
- measured (3)

→ Độ xác thực của claim.

2. SourceRef

Nguồn dữ liệu (6 nguồn):

- local_binary
- hf_api
- hf_card
- local_config
- derived
- heuristic

3. Resolution

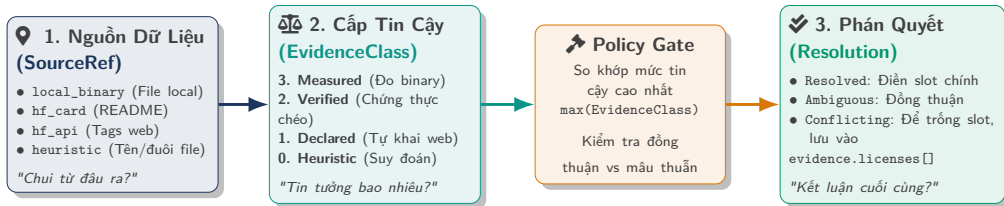
Trạng thái (4 loại):

- Resolved (Rõ ràng)
- Ambiguous (Đồng thuận)
- Conflicting (Tranh chấp)
- Unknown (Chưa rõ)

i Giá Trị Tính Toán Dẫn Xuất (Derived Fields)

$\text{VRAM} = \text{params} \times 2 \text{ bytes (BF16)}$ → tính trực tiếp từ giá trị đo đạc *measured* → giữ nguyên cấp tin cậy **Measured**.

Sơ Đồ Trực Quan: Luồng Phối Hợp 3 Trục Khái Niệm Bằng Chứng



💡 Tính Độc Lập Trục Giao (Orthogonality)

SourceRef chỉ nguồn gốc vật lý; **EvidenceClass** chỉ độ vững chắc khoa học; **Resolution** là kết quả logic sau khi đối chiếu. Tuyệt đối không đánh đồng nguồn gốc với kết luận.

Ví Dụ: Tại Sao Cùng 1 Request API Lại Tách 2 Nguồn Dữ Liệu?

```
{
  "id": "user/my-model",
  // 1. NGUON hf_api: Tac gia chon tren Web Settings
  "tags": ["license:apache-2.0", "text-generation"],

  // 2. NGUON hf_card: Boc tu ruot file README.md
  "cardData": {
    "license": "mit",
    "datasets": ["c4"]
  }
}
```

⚡ Nguy Cơ Mất Dữ Liệu Nếu Gộp Nguồn

- Hugging Face **không** tự động đồng bộ 2 trường này.
- Nếu gộp chung thành 1 nguồn hf: Giá trị apache-2.0 và mit sẽ đè mất nhau.
- **Giải pháp:** Tách riêng 2 nguồn hf_api (Tags) và hf_card (README frontmatter) → Engine tự động phát hiện mâu thuẫn nội bộ.

Phân Loại File & Giới Hạn Mức Tin Cậy (FileRole)

Vai Trò File	Mức Tin Cậy Max	Tạo Candidate?	Nguyên Tắc Xử Lý
Weight	measured	Có (Đầy đủ)	File chứa trọng số model → match hash xác định danh tính model.
Auxiliary	heuristic	Có (Bị chặn)	File cấu hình dùng chung → khóa trần heuristic tránh gán nhầm.
Media	N/A	Không	File ảnh, icon → 0 dùng để nhận diện model.
Archive	N/A	Không	File nén .zip, .tar → 0 dùng để nhận diện model.

Ví Dụ Trực Quan: Tại Sao Bắt Buộc Phân Loại FileRole?

1. File Weight (`model.safetensors`)

- Trọng số 16 GB chứa cấu trúc tensor.
- Match SHA-256 → Xác định đúng model Meta-Llama-3-8B (Cấp Measured).

3. File Media (`logo.png` 300 byte)

- Xuất hiện ở **352 repos** khác nhau.
- Từ chối tạo candidate model → chỉ xuất component type: "file".

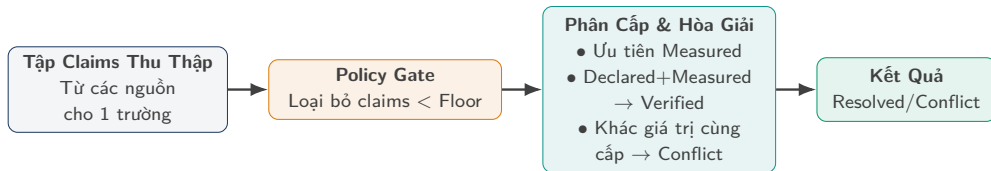
2. File Phụ Trợ (`tokenizer.json`)

- Dùng chung ở **1,037 repos** với 12 license.
- Khóa trần ở mức heuristic → không cho phép tự ý gán purl model cấp cao.

4. File Archive (`weights.zip`)

- File nén nhiều định dạng bên trong.
- Không dùng mã hash archive để nhận diện danh tính model.

Cách Xử Lý Mâu Thuẫn Trong Cùng 1 Repo (from_claims)



🛡️ Lọc Bằng Chứng Rác Bằng Policy Gate

Đặt sàn = *declared* → loại bỏ phỏng đoán *heuristic* trước khi so sánh → chống báo động mâu thuẫn giả.

6 Tình Huống Xử Lý Thực Tế Của Thuật Toán from_claims

Tình Huống	Tập Claims Đầu Vào	Floor	Kết Quả Phán Quyết
A. Đồng thuận	MIT (Card) + MIT (API)	heuristic	Resolved { MIT, declared }
B. Xác thực chéo	4096 (Card) + 4096 (Header đo)	heuristic	Resolved { 4096, verified }
C. Đo lường thẳng	FP32 (Card) + BF16 (Header đo)	heuristic	Resolved { BF16, measured }
D. Xung đột nội bộ	MIT (Card) + Apache-2.0 (API)	heuristic	Conflicting (Để trống)
E. Gate lọc rác	MIT (Card) + GPL (Tên phỏng đoán)	declared	Resolved { MIT, declared }
F. Không có dữ liệu	Trống 0 có claim nào	declared	Unknown

Cách Xử Lý Khi 1 File Trùng Ở Nhiều Repo (across_candidates)

1. Đánh Giá Tranh Chấp Trên Toàn Bộ Repo

- Duyệt và kiểm tra mâu thuẫn trên toàn bộ N repos trước khi cắt giảm danh sách.
- 100% repo cùng license → Ambiguous(agreed) (điền license).
- Có repo lệch license → Conflicting (để trống license chính).

2. Thuật Toán Round-Robin Giữ Đủ License

- Thuật toán Round-Robin: Giới hạn 50 repo mẫu đại diện theo từng nhóm license.
- Lấy luân phiên đại diện từng nhóm.
- Đảm bảo giữ đủ 12/12 loại license xuất hiện.

⚡ Thách Thức: Lệch Tần Suất Dữ Liệu

- File tokenizer.json xuất hiện ở **1,037 repos** với 12 loại license.
- Các repo phổ biến nhất chỉ tập trung vào MIT và Apache-2.0.
- Nếu chỉ lấy mẫu theo số lượng → bỏ sót 10 loại license còn lại (GPL, BSD, CC-BY...)?

✓ Giải Pháp: Phân Bỏ Đại Diện Round-Robin

- Gom 1,037 repos vào 12 nhóm theo từng loại license.
- Nhật luân phiên: 1 MIT, 1 Apache, 1 GPL, 1 BSD...
- Lặp lại cho đến khi đủ 50 repo đại diện.
- → Đảm bảo **100% độ bao phủ cho 12/12 loại license!**

12 Tình Huống Thực Tế Khi Quét Model & JSON Mẫu

Tổng Hợp 12 Case Thực Tế Khi Quét Model

Ca	Tình Huống Thực Tế	Cấu Trúc File	Trạng Thái	Hành Vi JSON Đầu Ra
1	Khớp Đơn Nhất 1-1	1 File ↔ 1 Repo	Resolved	Điền đủ Native slots, confidence: 1.0
2	Nhiều Repo Đồng Thuận	1 File → 3 Repos (cùng license)	Ambiguous(agreed)	Điền Native slot licenses, candidateCount: 3
3	Nhiều Repo Lỗi License	1 File 43 GiB → 3 Repos (khác license)	Conflicting	Để trống slot chính thức, ghi nhận evidence
4	File Phụ Trùng Hàng Nghìn Repo	tokenizer.json → 1,037 Repos	Conflicting	Round-Robin (50 đại diện cover 12 licenses)
5	File Media Không Phải Model	logo.png → 352 Repos	FileRole::Media	0 tạo candidate model, 0 xuất purl
6	Thư Mục Model Chia Nhiều Shards	3 Shards + config.json	Resolved(verified)	Gom nhóm weights, thăng cấp Verified khi khớp config
7	Thư Mục Chứa Nhiều Bản Quant	7 GGUFs + 1 config.json	Multi-Group	Root chuyển thành type: "file", ẩn shape vào withheld
8	File Chưa Từng Được Crawl	Hash Miss (File nội bộ)	Unknown	Sniff binary header → xuất parameters ở cấp measured
9	Mâu Thuẫn Trong Cùng 1 Repo	hf_api (Apache) vs hf_card (MIT)	Conflicting	Để trống slot chính thức, phơi bày cả 2 trong evidence
10	Khai Báo Khác Đo Đặc Thực Tế	Card khai 70B vs Header đo 6.74B	Resolved(measured)	Measured thăng tuyệt đối, log audit trail
11	File Weights 0-Byte Rác	File .bin 0 byte	EMPTY_SHA256	Loại bỏ mã băm rỗng, cảnh báo không có file hợp lệ
12	File ONNX Kèm File Tách Rời	model.onnx + _data_N	Resolved(sharded)	Gom nhóm các file phân tách vào cùng 1 model group

Ca 1: Khớp Đơn Nhất 1–1 (Single Model Match)

✓ Đặc Tính & Cách Xử Lý

- File `model.safetensors` khớp đúng 1 repo.
- Đủ bằng chứng → điền full native slots.
- Khớp SHA-256 tuyệt đối với cơ sở dữ liệu → gán độ tin cậy `confidence: 1.0`.

```
{
  "bomFormat": "CycloneDX", "specVersion": "1.6",
  "serialNumber": "urn:uuid:3e6717a6-...",
  "components": [{
    "bom-ref": "model-scan-001",
    "type": "machine-learning-model",
    "name": "Meta-Llama-3-8B", "group": "meta-llama",
    "purl": "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1",
    "licenses": [{"license": {"name": "llama3"}}],
    "evidence": {
      "identity": {
        "field": "purl", "confidence": 1.0,
        "methods": [{"technique": "hash-comparison", "confidence": 1.0}]
      }
    }
  ]
}
```

Ca 2: Nhiều Repo Nhưng Cùng Khai Báo (Multi-Repo Unanimous)

Đặc Tính & Cách Xử Lý

- File weights nằm ở 3 mirror repos.
- Cùng khai license: llama3 và tác giả meta-llama.
- Ambiguous(agreed) → vẫn điền native slot licenses.

```
{  
  "components": [{  
    "bom-ref": "model-scan-002",  
    "type": "machine-learning-model",  
    "name": "Llama-3-8B",  
    "licenses": [{"license": {"name": "llama3"}}],  
    "evidence": {  
      "identity": {  
        "field": "purl",  
        "candidateCount": 3,  
        "resolution": "ambiguous",  
        "agreement": true,  
        "methods": [{"technique": "hash-comparison", "confidence": 1.0}]  
      }  
    }  
  ]  
}
```

⚠ Đặc Tính & Cách Xử Lý

- File 43 GiB nằm ở 3 repos (Repo A: *other*, Repo B: *MIT*, Repo C: *trống*).
- Slot licenses: [] để trống → 0 phán bừa.
- Đẩy 3 claims vào `evidence.licenses[]`.

```
{
  "components": [{
    "type": "machine-learning-model", "name": "SomeModel",
    "licenses": [], // De trong do bat dong thong tin
    "evidence": {
      "identity": {
        "resolution": "conflicting", "conflicts": ["license"]
      },
      "licenses": [
        {"license": {"name": "other"}, "source": "repo-a", "downloads": 5200000},
        {"license": {"id": "MIT"}, "source": "repo-b", "downloads": 200000}
      ]
    }
  ]
}
```

Ca 4: File Phụ Trùng Hàng Nghìn Repo (tokenizer.json)

🔄 Đặc Tính & Cách Xử Lý

- File tokenizer.json xuất hiện ở 1,037 repos.
- Khóa trồn ở mức *heuristic* → 0 gán nhầm danh tính.
- Thuật toán Round-Robin lấy 50 đại diện giữ đủ 12 loại license.

```
{
  "components": [{
    "name": "tokenizer", "licenses": [],
    "evidence": {
      "identity": {
        "candidateCount": 1037,
        "resolution": "conflicting", "conflicts": ["license"]
      },
      "licenses": [
        // 50 đại diện Round-Robin cân bằng từ 12 nhóm license
        {"license": {"id": "Apache-2.0"}, "source": "repo-1"},
        {"license": {"id": "MIT"}, "source": "repo-2"},
        {"license": {"id": "GPL-3.0"}, "source": "repo-3"}
        // Bao phủ đầy đủ 12 nhóm license
      ]
    }
  ]
}
```

⊘ Đặc Tính & Cách Xử Lý

- File logo.png 300 byte trùng ở 352 repos.
- Nhận diện là Media → 0 tạo candidate model.
- Output ra type: "file", 0 gán purl model.

```
{  
  "components": [{  
    "bom-ref": "file-scan-005",  
    "type": "file",  
    "name": "huggingface_logo.png",  
    "hashes": [{  
      "alg": "SHA-256",  
      "content": "def456..."  
    }]  
    // Không gán purl và modelCard  
    // Tu chời suy diên sai danh tính model  
  }]  
}
```

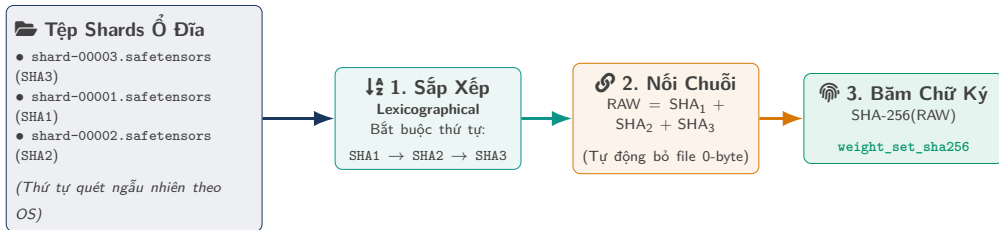

Ca 6: Thư Mục Model Chia Nhiều Shards

Đặc Tính & Cách Xử Lý

- Thư mục chứa 3 shards
.safetensors + config.json.
- Gom 3 shards → tính
weight_set_sha256.
- Đối chiếu với config.json →
thăng cấp lên **Verified**.

```
{
  "metadata": {
    "component": {
      "type": "machine-learning-model",
      "name": "Meta-Llama-3-8B",
      "purl": "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1"
    }
  },
  "components": [{
    "modelCard": {
      "modelParameters": {
        "architecture": "LlamaForCausalLM",
        "parameters": 8030000000 // Dat cap Verified qua cross-verify
      }
    }
  ]
}
```

Ví Dụ Trực Quan: Cách Gom Nhóm & Tính Chữ Ký Shards



✓ Tính Tất Định Tuyệt Đối (Determinism)

Cho dù hệ điều hành (Windows NTFS, Linux ext4, macOS APFS) liệt kê file theo thứ tự nào, bước **Lexicographical sort** đảm bảo chuỗi băm RAW luôn giống nhau → Chữ ký `weight_set_sha256` trùng khớp byte-for-byte trên mọi hệ thống.

Ca 7: Thư Mục Chứa Nhiều Bản Lượng Tử Hóa (GGUF Đa Nhóm)

📁 Đặc Tính & Cách Xử Lý

- Thư mục chứa 7 file GGUF multi-quant + 1 config.json.
- Thư mục đa model → Root chuyển thành type: "file".
- Ẩn các trường hình thái model vào danh sách withheld.

```
{
  "metadata": {
    "component": {
      "type": "file", // Chuyển từ machine-learning-model
      "name": "my_models_dir"
    }
  },
  "properties": [
    {"name": "aibom:withheld", "value": "architecture,param_count,context_length"}
  ]
  // 7 cum model con duoc mo ta doc lap trong envelope report/2
}
```

Ví Dụ Trực Quan: Xử Lý Thư Mục 7 Bản Lượng Tử Hóa GGUF

📁 **Thư Mục my_models/**

- ├ llama-3-q2_k.gguf (4 GB, Q2)
- ├ llama-3-q4_k_m.gguf (6 GB, Q4)
- ├ llama-3-q8_0.gguf (10 GB, Q8)
- ├ ... (tổng cộng 7 bản quant)
- └ config.json (**Ghi shape FP16!**)

⊗ **Nguy Cơ Áp Nhầm Config**

- Nạp config.json vào root
- Gán thông số FP16 (32 GB VRAM) cho cả file Q2_K (chỉ cần 4 GB VRAM)
- → **Sai lệch thông số phần cứng!**

🛡️ **Giải Pháp Cách Ly AI-BOM**

- Root chuyển thành type: "file"
- Ẩn shape chung vào withheld
- Tách 7 bản GGUF thành 7 **components con**

với thông số đo đạc độc lập!

🛡️ **Nguyên Tắc An Toàn Metadata**

Khi một thư mục chứa nhiều model có cấu hình khác nhau, tuyệt đối không cho phép file cấu hình ngoài (như config.json) áp đặt thuộc tính chung lên toàn bộ thư mục.

🔍 Đặc Tính & Cách Xử Lý

- File binary nội bộ chưa có trên Hugging Face.
- Danh tính → resolution: unknown.
- Sniff header file để đo trực tiếp thông số ở cấp **Measured**.

```
{
  "components": [{
    "type": "machine-learning-model",
    "name": "unknown-model",
    "modelCard": {
      "modelParameters": {
        "parameters": 6738415616, // Do trực tiếp từ header (Measured)
        "dataType": "BF16" // Do trực tiếp từ header (Measured)
      }
    },
    "evidence": {
      "identity": {"resolution": "unknown"}
    }
  }]
}
```

Cả 9: Mâu Thuẫn Trong Cùng 1 Repo

⚡ Đặc Tính & Cách Xử Lý

- Cùng 1 repo: API khai Apache-2.0 nhưng Card khai MIT.
- Tách riêng từng nguồn dữ liệu → phát hiện mâu thuẫn.
- Để trống slot chính, lưu cả 2 nguồn vào evidence.

```
{
  "components": [{
    "name": "my-model", "licenses": [],
    "evidence": {
      "identity": {"resolution": "conflicting", "conflicts": ["license"]},
      "licenses": [
        {"license": {"id": "Apache-2.0"}, "source": "hf_api", "class": "declared"},
        {"license": {"id": "MIT"}, "source": "hf_card", "class": "declared"}
      ]
    }
  ]
}
```

🔍 Đặc Tính & Cách Xử Lý

- Model Card khai nhầm:
param_count: 70B.
- Sniff header tensor đếm ra: **6.74B** params.
- **Measured > Declared**: Chốt 6.74B, ghi log audit.

```
{
  "components": [{
    "modelCard": {
      "modelParameters": {
        "parameters": 6738415616 // Chốt theo kết quả đo lường
      }
    },
    "properties": [
      // Lỗi khai 70B bị loại và lưu vào audit log
      {"name": "aibom:audit.rejected_claim", "value": "param_count:70B (hf_card)"}
    ]
  }]
}
```

Ca 11: File Weights 0-Byte (Loại Bỏ Mã Băm Rỗng)

! Đặc Tính & Cách Xử Lý

- Thư mục chứa file weights 0 byte.
- Loại bỏ hash rỗng khỏi nhận diện model.
- Báo warning "no valid weight files", 0 match bù.

```
{
  "metadata": {
    "component": {"type": "file", "name": "empty_model_dir"}
  },
  "properties": [
    {
      "name": "aibom:warning",
      "value": "No valid weight files found. Directory does not identify a model."
    }
  ]
}
```


Đặc Tính & Cách Xử Lý

- Model ONNX chia `model.onnx` + `model.onnx_data_N`.
- Tự động cắt hậu tố số → gom 245 file split (332 GB) vào 1 weight group.

```
{
  "components": [{
    "type": "machine-learning-model",
    "name": "gemma-4-onnx",
    "properties": [
      {"name": "aibom:weight_set_sha256", "value": "a1b2c3..."},
      {"name": "aibom:group_members", "value":
        "model.onnx,model.onnx_data_1,model.onnx_data_2"}
    ]
  }]
}
```

Cấu Trúc JSON Đầu Ra: Chuẩn Quốc Tế & Nội Bộ

8 Quy Tắc Quyết Định Dữ Liệu Xuất Ra JSON

#	Tên Quy Tắc	Hành Vi Xuất Dữ Liệu Thực Tế
1	Native Slot Gating	Mâu thuẫn → để trống slot chính, đẩy claims vào evidence.
2	Structural Override	Kết quả đo measured thắng tuyệt đối lời khai declared.
3	FileRole Ceiling	File phụ khóa trần ở heuristic; Media 0 tạo candidate.
4	Round-Robin Coverage	Giới hạn 50 repo mẫu nhưng giữ đủ đại diện 12 loại license.
5	Full Candidate Check	Kiểm tra mâu thuẫn trên toàn bộ candidate trước khi cắt giảm.
6	Policy Gate	Lọc claims < floor trước khi hòa giải để chống mâu thuẫn giả.
7	Multi-Group Withholding	Thư mục đa nhóm: Root là file, đưa thông số model vào withheld.
8	SPDX Auto-Routing	Khớp SPDX → license.id; Không khớp → license.name.

1. Chuẩn Quốc Tế CycloneDX 1.6

- Chuẩn mở ML-BOM của OWASP.
- Tương thích ngay với các tool kiểm toán và SIEM quốc tế.
- Định danh components, model parameters, datasets.

2. Định Dạng Đầy Đủ report/2

- Định dạng mở rộng nội bộ phục vụ audit sâu.
- Phân rõ chi tiết từng nhóm weights (groups []).
- Lưu near_misses[], withheld[] và completeness score.

Ví Dụ: Cấu Trúc Báo Cáo Nội Bộ Envelope (report/2)



Đặc Điểm Tầng Envelope

- **Hai tầng phân cấp:** Tầng ngoài bọc báo cáo audit nội bộ (report/2), tầng trong chứa payload CycloneDX 1.6.
- **Theo dõi cụm weights:** Mảng groups[] theo dõi từng nhóm weights và chữ ký weight_set_sha256.
- **Minh bạch nguồn gốc:** Trường provenance ghi rõ từng thuộc tính lấy từ đâu.
- **Chấm điểm dữ liệu:** Trường completeness_score đánh giá 40 tiêu chí chất lượng.

```
{
  "schema": "opswat.aibom.report/2",
  "engine_version": "0.1.0",
  "scan_target": "/data/models/Meta-Llama-3-8B",
  "scan_timestamp": "2026-09-01T12:00:00Z",
  "deterministic": true,
  "completeness_score": 0.92,
  "groups": [{
    "group_name": "Meta-Llama-3-8B-Weights",
    "weight_set_sha256": "4a7d8b2e1f9c3a0b...",
    "member_count": 4,
    "total_bytes": 16060522496,
    "resolution": "resolved",
    "candidates": [
      "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1"
    ],
    "provenance": {
      "license": "hf_card",
      "param_count": "local_binary",
      "architecture": "local_config"
    }
  ]},
  "cyclonedx": { /* CycloneDX 1.6 ML-BOM Payload */ }
}
```

Ảnh Xạ Dữ Liệu Vào Chuẩn Quốc Tế CycloneDX 1.6

Trường CycloneDX 1.6 Chuẩn	Nguồn Dữ Liệu & Ảnh Xạ Hệ Thống
hashes[]	SHA-256 từng file và chữ ký weight_set_sha256
name / group	Tên model / Namespace tác giả (<i>meta-llama</i>)
version / purl	Commit SHA / Purl chuẩn pkg:huggingface/<author>/<name>@<sha>
licenses[]	License kết luận (SPDX ID hoặc Custom Name)
modelCard.modelParameters	architectureFamily, modelArchitecture, task, parameters, datasets[]
evidence.identity	Method định danh (<i>hash-comparison</i> / <i>binary-analysis</i>), confidence: 1.0
evidence.licenses[]	Danh sách claims khi có tranh chấp license
Thuộc Tính Mở Rộng ('aibom:*' / 'lms:*')	Ý Nghĩa Kỹ Thuật Chuyên Biệt
aibom:quantization	Kiểu lượng tử hóa (<i>Q4_K_M</i> , <i>Q8_0</i> , <i>FP16</i>)
lms:param_count / lms:tensor_count	Tổng params và tổng tensors đếm trực tiếp từ header file
lms:context_length / lms:layer_count	Context length tối đa và số layer kiến trúc
aibom:withheld	Danh sách trường bị ẩn để tránh sai lệch trong thư mục đa model

Ví Dụ: Payload Chuẩn Quốc Tế CycloneDX 1.6 ML-BOM

✓ Chuẩn Hóa CycloneDX 1.6

- **UUIDv8 tất định:** serialNumber theo RFC 9562, khóa timestamp → byte-for-byte.
- **Không trùng lặp:** metadata.component không bom-ref tránh xung đột unique.
- **Khả định danh tính:** confidence: 1.0 tuyệt đối với băm SHA-256.
- **14 Extensions:** Đo đặc thực tế (lms:*) và lượng tử hóa (aibom:*)

```
{
  "bomFormat": "CycloneDX", "specVersion": "1.6",
  "serialNumber": "urn:uuid:3e6717a6-2009-8786-9051-7f888eb1a57e",
  "components": [{
    "bom-ref": "model-artifact-root", "type": "machine-learning-model",
    "name": "Meta-Llama-3-8B",
    "purl": "pkg:huggingface/meta-llama/Meta-Llama-3-8B@e9149a1",
    "licenses": [{ "license": { "name": "llama3" } }],
    "modelCard": {
      "modelParameters": {
        "architectureFamily": "llama",
        "parameters": 8030261248 // Measured
      }
    },
    "evidence": {
      "identity": {
        "field": "purl", "confidence": 1.0,
        "methods": [{ "technique": "hash-comparison", "confidence": 1.0 } ]
      }
    },
    "properties": [
      { "name": "aibom:quantization", "value": "BF16" },
      { "name": "lms:param_count", "value": "8030261248" }
    ]
  }]
}
```

⚠ Yêu Cầu Của Schema CycloneDX 1.6

Validator sẽ báo lỗi nếu `license.id` chứa giá trị nằm ngoài từ điển SPDX.

✓ Tự Động Điều Hướng 811 SPDX IDs

- Khớp từ điển SPDX → ghi vào `license.id`.
- Tên riêng tự đặt (llama3) → ghi vào `license.name`.
- 100% pass CycloneDX schema validator.

🌀 Sinh ID Tất Định Theo RFC 9562 UUIDv8

- **UUIDv4 vs UUIDv8:** UUIDv4 là ngẫu nhiên; UUIDv8 cho phép nhúng payload tùy biến (SHA-256 + timestamp cố định).
- `serialNumber` sinh tất định từ 16 byte đầu SHA-256 của file.
- Khóa timestamp → quét 2 lần cùng 1 target cho output JSON trùng byte-for-byte.

Đánh Giá Độ Đầy Đủ Của Metadata (Completeness Score)

📝 40 Trường Đánh Giá

Kiểm tra 4 nhóm thông tin chính:

- **Danh tính:** name, author, purl, sha256
- **Pháp lý:** license, license_type
- **Thông số kỹ thuật:** param_count, dtype, architecture, context_length
- **Nguồn gốc:** base_model, datasets

% Công Thức Tính Completeness Score

$$\text{completeness_score} = \frac{\sum (\text{field có data} \times \text{weight})}{\sum \text{tổng weight}}$$

- Đo lường độ đầy đủ thông tin của model trong BOM.
- Tự động cảnh báo khi model thiếu license hoặc thông số quan trọng.

Đọc Header Binary, Chống Mã Độc & Kết Quả Test

Kiến Trúc Quét 2 Giai Đoạn (Two-Phase Scan Pipeline)

Phase 1: Fast Structural Sniffing (< 5ms)

Mục tiêu: Triage nhanh & Trích xuất hình thái

- Đọc vài KB đầu file (header length + JSON metadata).
- Trích xuất cấu trúc tensor: params, dtype, context length (Cấp Measured).
- Duyệt opcode tĩnh file Pickle → loại trừ mã độc RCE.
- Tiêu tốn < 50MB RAM, thực thi trong < 5ms.

Phase 2: Streaming Identity Hashing

Mục tiêu: Tính SHA-256 tra cứu Cache L2

- Đọc tuần tự theo buffer I/O chunking (64KB / 1MB).
- Tính SHA-256 từng file & weight_set_sha256 mà 0 nạp toàn bộ file vào RAM.
- Tra cứu exact hash vào Layer 2 SQLite (< 1ms).
- Quét file 40 GB an toàn, 0 tràn bộ nhớ (OOM).

Đọc Header File Binary Cực Nhanh (< 5ms, Không Load Weights)

Định Dạng	Module Rust	Cách Sniff Header Tốc Độ Cao (< 5ms)
GGUF	<code>formats/gguf.rs</code>	Đọc magic bytes GGUF, parse các cặp KV (<i>architecture</i> , <i>context_length</i> , quantization <i>Q4_K_M</i>).
Safetensors	<code>formats/safetensors.rs</code>	Đọc 8 bytes đầu lấy độ dài header, parse JSON metadata và danh sách tensor shapes/dtypes.
ONNX	<code>formats/onnx.rs</code>	Parse protobuf header: graph nodes, initializer tensor shapes.
HDF5	<code>formats/hdf5.rs</code>	Duyệt cây B-Tree / Group header trích xuất Keras weights.

Ví Dụ Trực Quan: Đọc Header File 40 GB Trong Dưới 5ms

✘ Cách Truyền Thống (Load Toàn Bộ)

- Dùng Python/PyTorch load toàn bộ file weights 40 GB vào RAM.
- Thời gian thực thi: **30–60 giây**.
- Tiêu tốn > 40 GB RAM, dễ crash/OOM khi scan thư mục lớn.

⚡ Cơ Chế Binary Sniffer Của AI-BOM

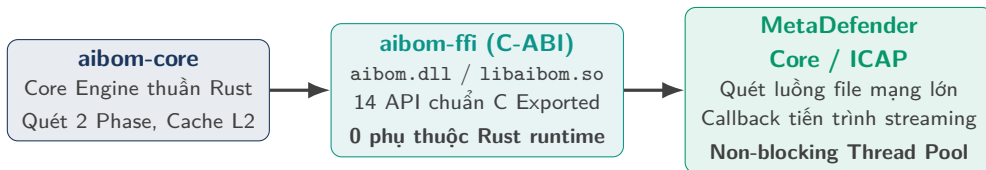
- Đọc đúng **8 bytes đầu** để lấy độ dài header (N bytes).
- Đọc tiếp N bytes JSON metadata của header.
- Thời gian thực thi: < 5 ms.
- Tiêu tốn < 50 MB RAM, trích xuất đủ tensor shapes và dtypes.

⚠️ Rủi Ro Mã Độc Trong File Pickle

File PyTorch Pickle có thể chứa bytecode độc hại → rủi ro thực thi mã tùy ý (RCE) nếu nạp bằng Python interpreter chuẩn.

🔒 Bộ Phân Tích Opcode Tĩnh Trong `formats/pickle.rs`

- Máy trạng thái duyệt từng opcode nhị phân tĩnh trong file pickle.
- 0 khởi tạo Python runtime, 0 nạp đối tượng thực thi.
- Trích xuất an toàn tensor keys → **Triệt tiêu nguy cơ thực thi mã độc (RCE).**



🔧 Đặc Điểm Nhúng Vào Scanning Engine

- **C-ABI tương thích trực tiếp**: Nhúng dạng shared library vào C++, C#, Go, Python của MetaDefender.
- **Xử lý file lớn qua mạng (ICAP)**: Hỗ trợ streaming hash chunk-by-chunk với progress callback → không phong tỏa thread pool chính khi tải file model 50 GB.

Kết Quả Test Suite (366 Tests) & Benchmark Hiệu Năng

☰ Kết Quả Test Suite

Chạy tự động qua `acceptance.ps1`:

1. **Workspace Tests**: 366 tests pass 100% (0 fail, 0 skip).
2. **Corpus Assertions**: 49,992 models, DDL 40 cột.
3. **Determinism**: So sánh byte-for-byte.
4. **CycloneDX 1.6**: JSON Schema Pass 100%.
5. **Regressions**: 8 tests chống lỗi hồi quy cho các trường hợp đặc biệt.
6. **Clippy Hygiene**: 0 warning (-D warnings).

🕒 Benchmark Hiệu Năng

- **Query hash SQLite**: < 1 ms.
- **Sniff header (file 40 GB)**: < 5 ms.
- **RAM runtime**: < 50 MB.
- **Offline Rebuild**: Rebuild 50k models trong vài phút với 0 request mạng.

4 Vấn Đề Kỹ Thuật Thực Tế Đã Xử Lý Triệt Để

#	Vấn Đề Gặp Phải	Giải Pháp Đã Xử Lý
1	Sót file ONNX bị chia nhỏ	Regex cắt hậu tố <code><digits></code> → gom 245 file split (332 GB) vào đúng model group.
2	Nhận diện sai file 0-byte	Loại bỏ mã hash <code>EMPTY_SHA256</code> khỏi danh tính weight set.
3	Tràn metadata thư mục đa model	Gỡ shape fields khỏi root component, đưa vào danh sách withheld.
4	4,950 model khai license "other"	Tách cột <code>license_name</code> & <code>license_link</code> → chuẩn hóa 3,466 models sang tên license chuẩn.

Giá Trị Cốt Lõi Của AI-BOM 0.1.0

1. **Mô hình 4 mức tin cậy:** Đo đặc cấu trúc nhị phân trực tiếp > lời khai web, bảo đảm tính xác thực khoa học.
2. **SQLite 2 tầng:** Quét 100% offline không phụ thuộc mạng, nén 50k models còn 18 KB/model (912 MB).
3. **Xử lý 12 tình huống thực tế:** 1-N fan-out, multi-quant, sharded weights, mâu thuẫn license.
4. **Chuẩn hóa quốc tế:** Hỗ trợ chuẩn CycloneDX 1.6 ML-BOM và format mở rộng nội bộ.

Roadmap Kế Tiếp

Hoàn thiện Type 1 Final → Batch 4 (Delta sync & Cache lifecycle) → Batch 5 (Dataset catalog) → Batch 6.1 (Verified score).



Thảo Luận & Q&A

AI-BOM Architecture & Schema Specification · Version 0.1.0